

Calcolatori Elettronici T
Ing. Informatica

Traccia soluzione
9 Gennaio 2017

Esercizio 1

Dispositivi e segnali presenti nel sistema:

RAM_32	mappata da FE000000h:FFFFFFFFh, 4 banchi da 8 MB
EPROM_128	mappata da 00000000h:07FFFFFFh, 4 banchi da 32 MB
CS_INPUT_PORT	mappato a 80000000h (lettura da INPUT_PORT)
CS_OUTPUT_PORT	mappato a 80000001h (scrittura verso OUTPUT_PORT)
CS_RD_INT_STATUS	mappato a 80000002h (lettura stato interrupt)
CS_OUTPUT_PORT_PLUS_1	mappato a CS_OUTPUT_PORT
CS_OUTPUT_PORT_PLUS_2	mappato a CS_OUTPUT_PORT

Segnali di decodifica di memorie, periferiche e segnali:

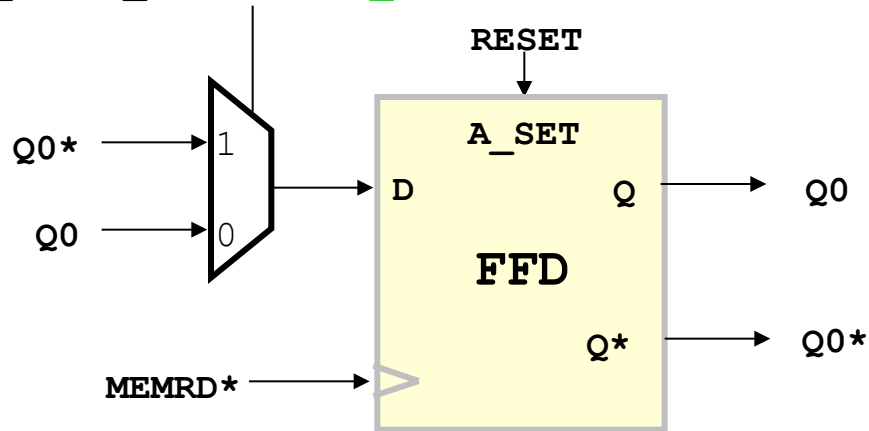
CS_RAM_32_0	=	BA31 · BA30 · BE0	
CS_RAM_32_1	=	BA31 · BA30 · BE1	
CS_RAM_32_2	=	BA31 · BA30 · BE2	
CS_RAM_32_3	=	BA31 · BA30 · BE3	
CS_INPUT_PORT	=	BA31 · BA30* · IBF · INPUT_ENABLED · BE0	; 80000000h
CS_OUTPUT_PORT	=	BA31 · BA30* · OBE · INPUT_ENABLED* · BE1	; 80000001h
CS_RD_INT_STATUS	=	BA31 · BA30* · MEMRD · BE2	; 80000002h
CS_OUTPUT_PORT_PLUS_1	=	CS_OUTPUT_PORT · READY_PLUS_1	
CS_OUTPUT_PORT_PLUS_2	=	CS_OUTPUT_PORT · READY_PLUS_2	
CS_EPROM_128_0	=	BA31* · BE0	
CS_EPROM_128_1	=	BA31* · BE1	
CS_EPROM_128_2	=	BA31* · BE2	
CS_EPROM_128_3	=	BA31* · BE3	

I segnali **INPUT_ENABLED**, **READY_PLUS_1** e **READY_PLUS2** presenti nei chip-select saranno sintetizzati nelle pagine successive. Il significato di tali segnali è il seguente:

- **INPUT_ENABLED** codifica se il trasferimento abilitato in un determinato istante è relativo a **INPUT_PORT** (**INPUT_ENABLED=1**) o **OUTPUT_PORT** (**INPUT_ENABLED=0**)
- **READY_PLUS_1** e **READY_PLUS2** codificano se è possibile trasferire il dato inviato in output anche (rispettivamente) in **OUTPUT_PORT_PLUS_1** e **OUTPUT_PORT_PLUS_2**

L'alternanza tra input e output può essere gestita in modo sincrono utilizzando due FFD come segue. Il condizionamento, nei MUX e nei CS, con il segnale **INPUT_ENABLED** è opzionale e serve unicamente a evitare dei malfunzionamenti causati da accessi impropri alle periferiche (i.e., non aderenti alle specifiche del problema). All'avvio, **INPUT_ENABLED=1**.

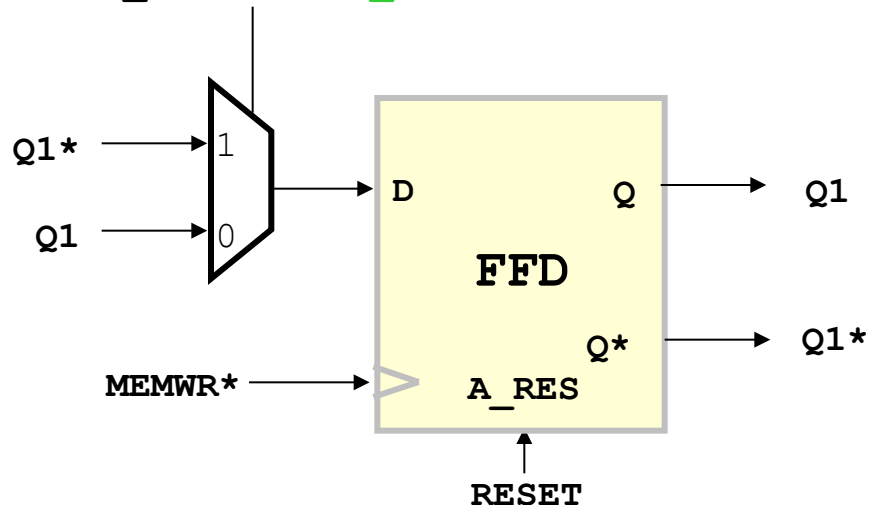
CS_INPUT_PORT · INPUT_ENABLED



Q0Q1 ATTIVO

1	0	INPUT	->	INPUT_ENABLED=1
0	0	OUTPUT	->	INPUT_ENABLED=0
0	1	INPUT	->	INPUT_ENABLED=1
1	1	OUTPUT	->	INPUT_ENABLED=0

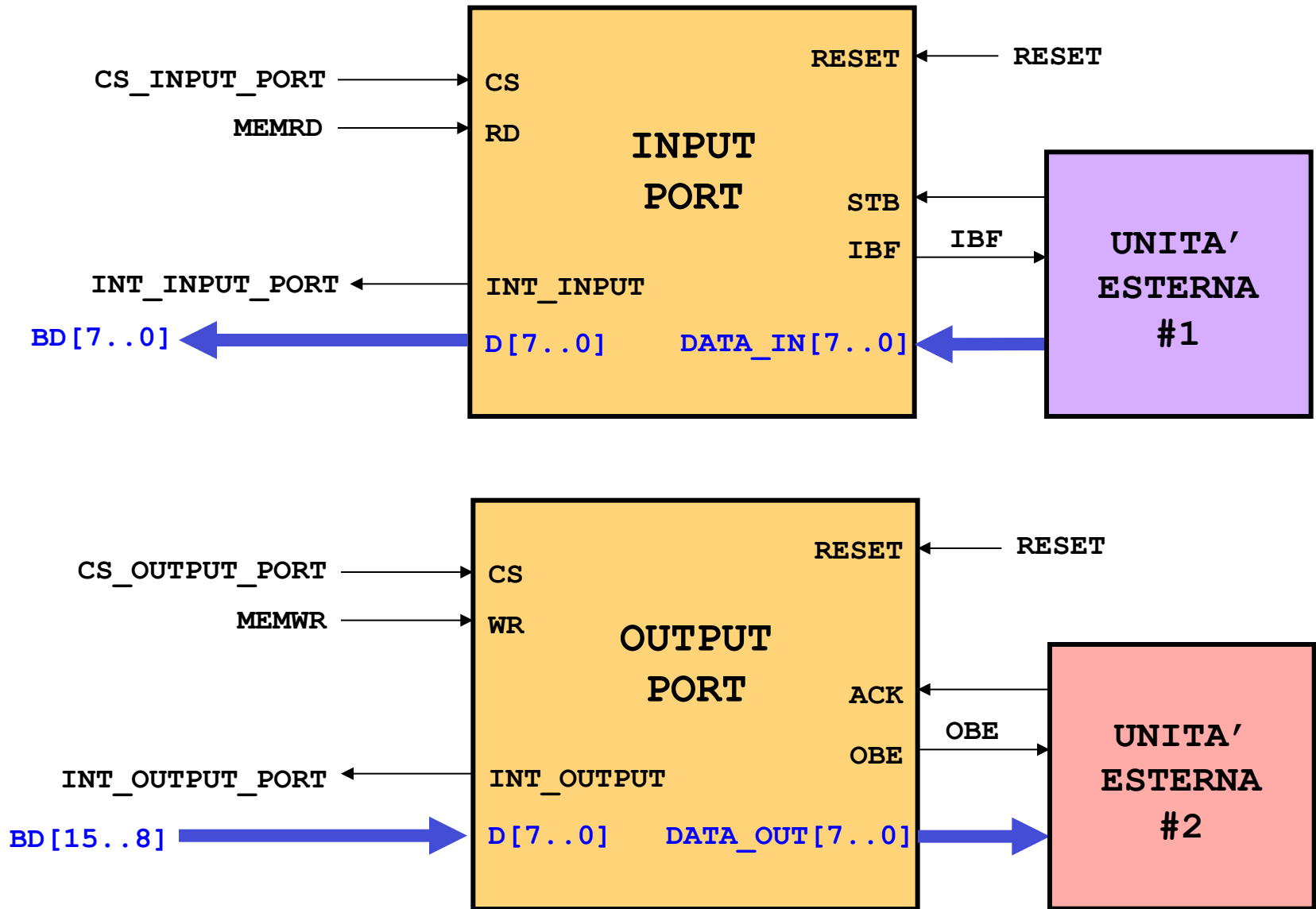
CS_OUTPUT_PORT · INPUT_ENABLED*



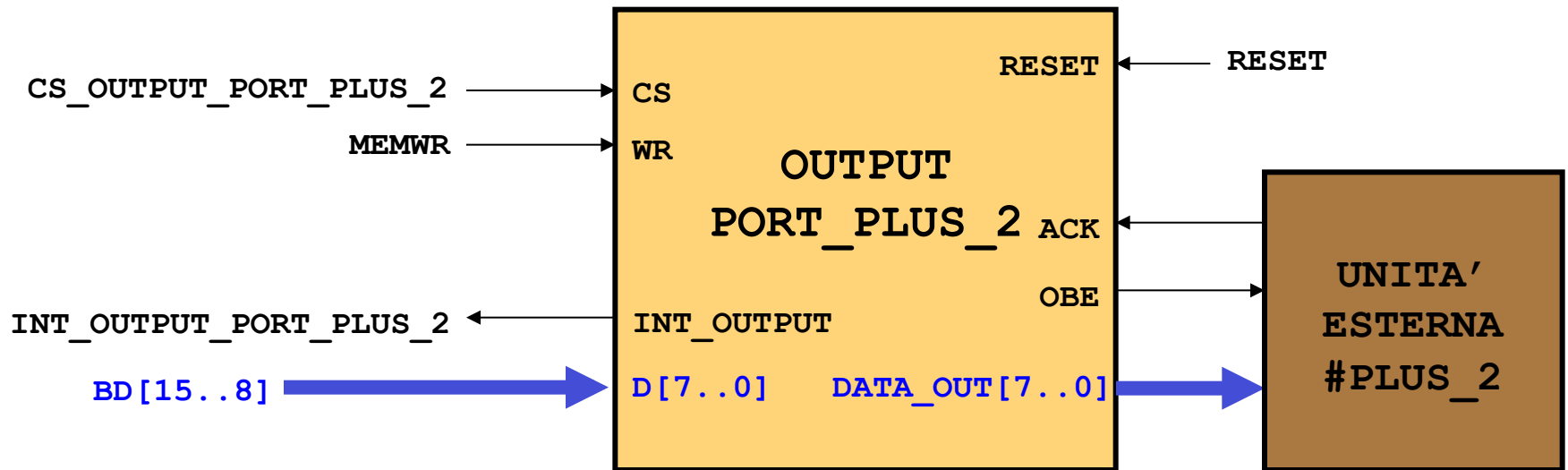
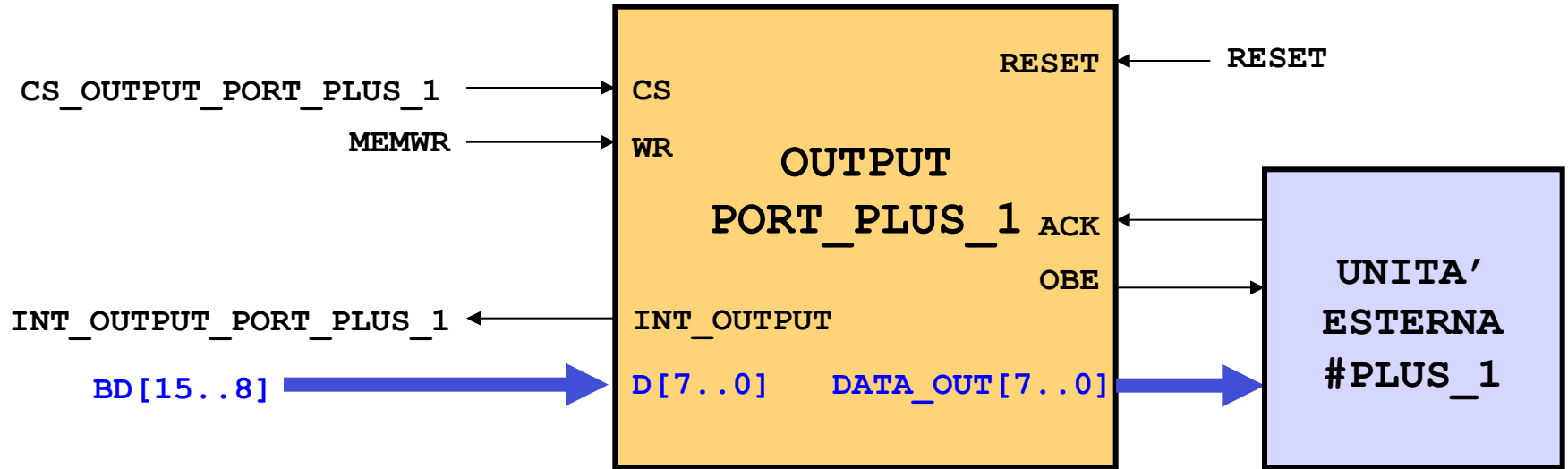
Pertanto,

$$\mathbf{INPUT_ENABLED} = Q0 \text{ XOR } Q1$$

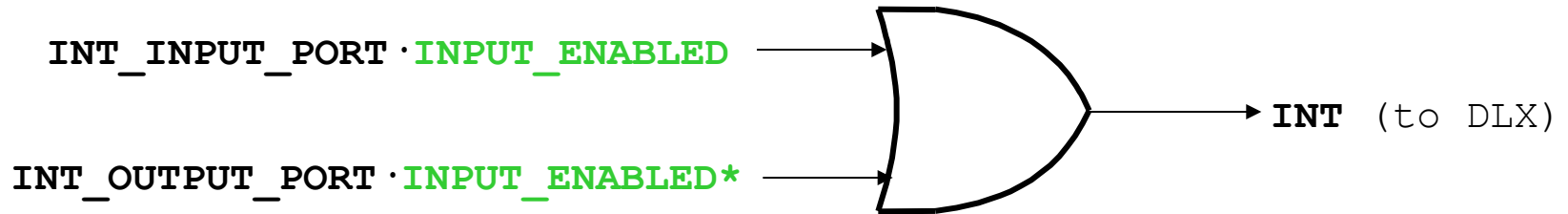
La porta in input è connessa al bus dati **BD[7..0]** mentre la porta in output è connessa a **BD[15..8]**.



Le altre due porte in output, OUTPUT_PORT_PLUS_1 e OUTPUT_PORT_PLUS_2, sono connesse, come OUTPUT_PORT, a **BD[15..8]**.



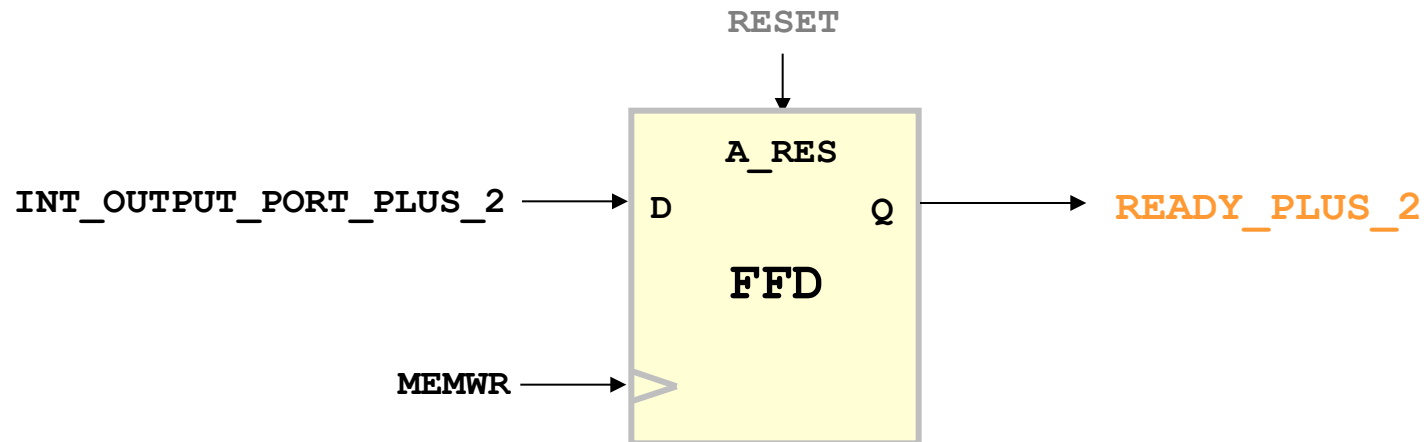
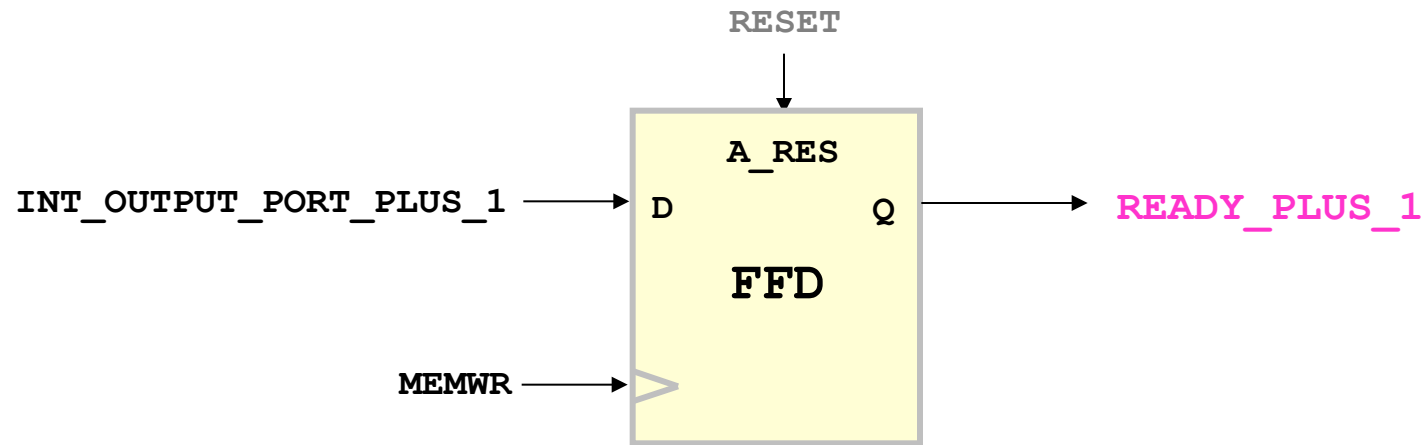
Il segnale **INPUT_ENABLED** sarà utilizzato per condizionare le richieste di interrupt generate da INPUT_PORT e OUTPUT_PORT al fine di garantire l'alternanza richiesta dalle specifiche del testo. Pertanto, il segnale di interrupt inviato al DLX risulta:



Inoltre, almeno uno dei due segnali di interrupt generati da INPUT_PORT e OUTPUT_PORT dovrà poter essere letto all'indirizzo **CS_READ_INT_STATUS** dall'interrupt handler al fine di poter determinare chi ha generato la richiesta di interruzione. La rete seguente consente di eseguire questa operazione. Si noti che in questo caso non è necessario un campionamento poiché non sussistono problemi di metastabilità nella lettura. Infatti, essendo gli interrupt mutuamente esclusivi, quando il segnale INT è a 1 l'unico segnale (i.e., uno dei due interrupt) che può averlo portato in tale stato non potrà cambiare durante la lettura eseguita dall'interrupt handler all'indirizzo CS_READ_INT_STATUS (i.e., 80000002h). I segnali **BD[23..17]** non sono pilotati; pertanto, in questo caso, nell'interrupt handler tali segnali dovranno essere opportunamente gestiti/mascherati.



La reti seguenti generano i segnali **READY_PLUS_1** e **READY_PLUS_2** che consentono, rispettivamente, di abilitare i trasferimenti del dato inviato a OUTPUT_PORT verso OUTPUT_PORT_PLUS_1 e OUTPUT_PORT_PLUS_2. Contrariamente al caso precedente, in questa circostanza, i segnali **INT_OUTPUT_PORT_PLUS_1** e **INT_OUTPUT_PORT_PLUS_2** sono campionati per evitare problemi di metastabilità durante il trasferimento in output.



Codice dell'interrupt handler:

```
LHI    R21,8000h           ; prepara indirizzo 80000000h
LBU    R22,2(R21)          ; legge (a 80000002h) lo stato dell'interrupt in R22
ANDI   R22,R22,1           ; maschera tutti i bit di R22 tranne il primo
BNEQZ  R22,input_port      ; se l'interrupt attivo è quello di INPUT_PORT
                                   ; (i.e., se R22=1) salta a input_port

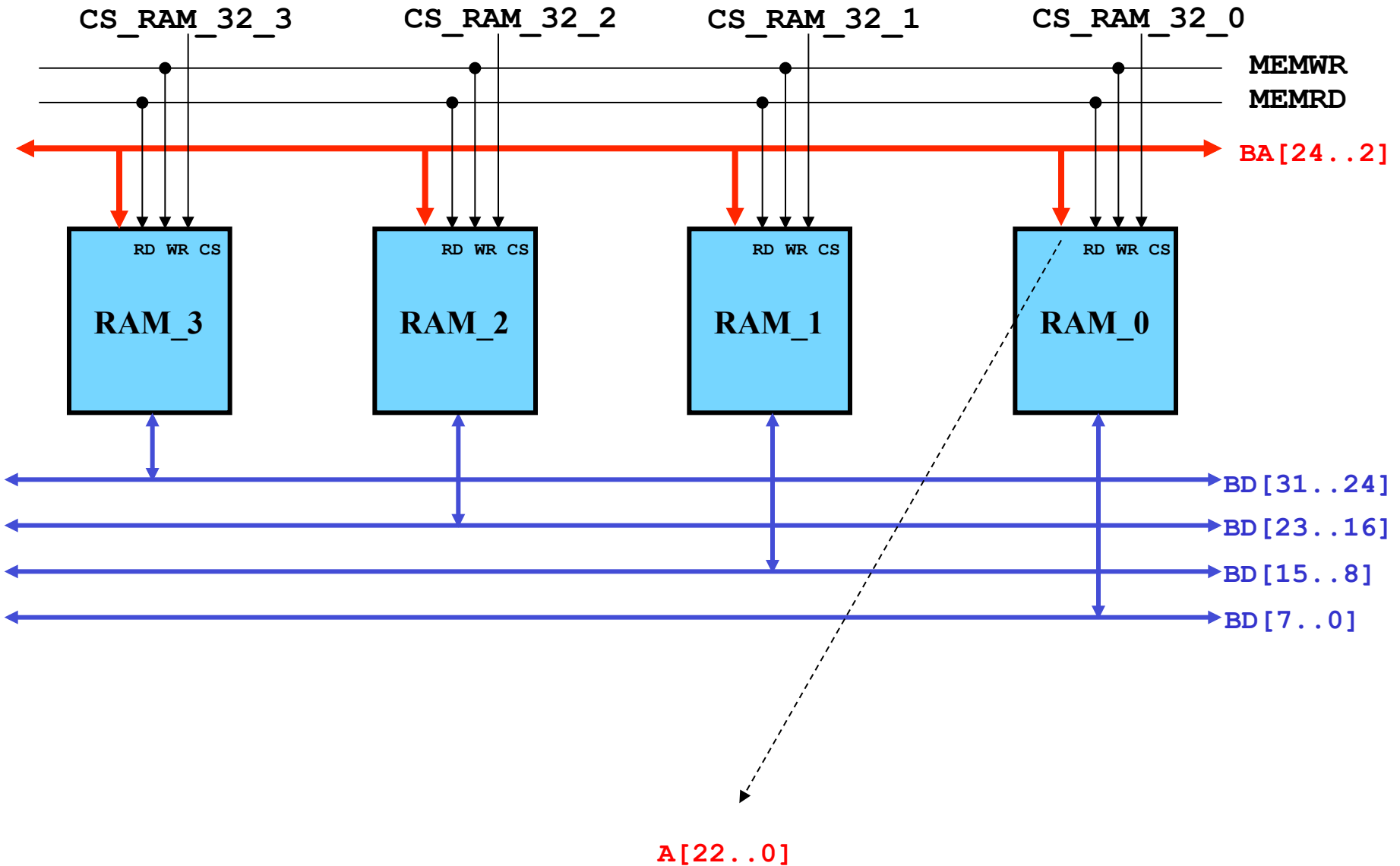
                                   ; se l'interrupt è relativo a OUTPUT_PORT (R22=0)
SB      R20,1(R21)          ; scrive il dato in R20 verso OUTPUT_PORT e, se questo è
                                   ; possibile, contemporaneamente anche nelle eventuali
                                   ; altre porte in output che risultano pronte
RFE                                   ; ritorna dall'interrupt

input_port:                  ; se l'interrupt è relativo a INPUT_PORT (R22!=0)
LBU     R20,0(R21)          ; trasferisce byte da INPUT_PORT (80000000h) a R20
RFE                                   ; ritorna dall'interrupt
```

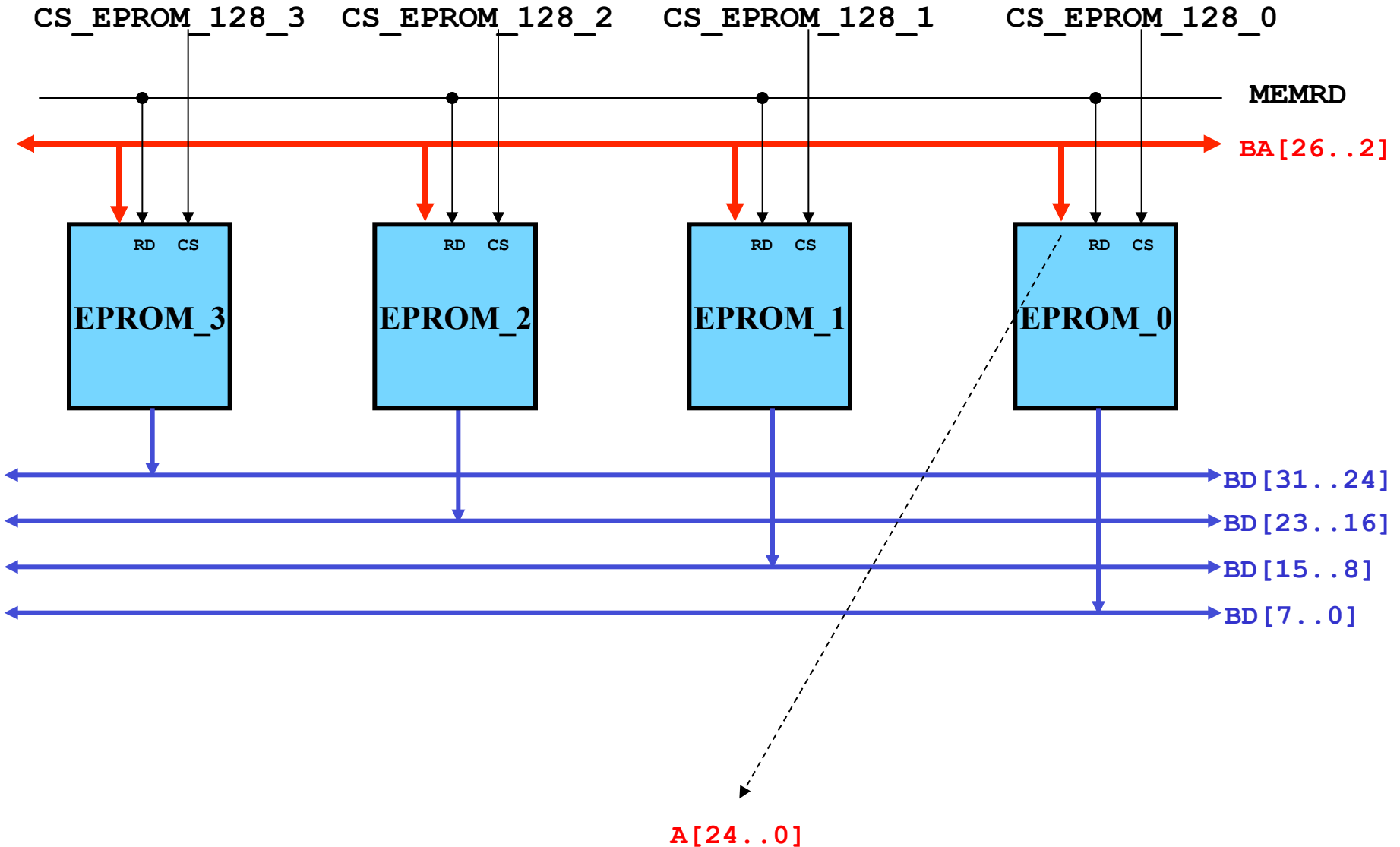
NOTE

- L'offset corrispondente alla label **input_port** è pari a +8
- L'istruzione **ANDI** potrebbe essere evitata gestendo opportunamente nel progetto i segnali connessi a **BD[23..17]** durante le letture a **CS_READ_INT_STATUS**

Interfacciamento RAM_32



Interfacciamento EPROM_128



Esercizio 2

Rif. lucidi/lezioni.

Esercizio 3

Rif. lucidi/lezioni.